# Práctico 07 — Terreno desde heightmap

## Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

9 de octubre de 2026



# Dónde quedamos y qué entra hoy

## Lo que quedó pendiente

De la clase anterior quedaron sin ver **la actividad del acumulador** y **las entradas de usuario**. Entran hoy, al principio, antes del terreno.

## Lo nuevo

La **malla de terreno** a partir de un heightmap, cómo se integra al simulador y cómo configurar OpenGL para el manejo de **eliminación de superficies ocultas**.

## Aviso

**Checkpoint de proyecto** el **viernes 27 de noviembre**. A esta altura el proyecto debería tener: la escena con la cámara orbit andando, la aeronave armada con jerarquía de matrices, el game loop con paso fijo y FDM integrado, entradas de teclado que interactuen con la escena, skybox / skydome y algo de texturas.

# Actividad aúlica: la tabla resuelta

$\Delta t$ del FDM $= 8{,}33$ ms (120 Hz)

| cuadro | `frame_dt` | antes | pasos | después |
|---|---|---|---|---|
| 1 | 16,7 ms | 0,00 | **2** | 0,04 |
| 2 | 33,4 ms | 0,04 | **4** | 0,12 |
| 3 | 250 ms | 0,12 | **30** | 0,22 |
| 4 | 8,0 ms | 0,22 | **0** | 8,22 |
| 5 | 8,0 ms | 8,22 | **1** | 7,89 |

## El invariante

37 pasos $\times\, 8{,}33 =$ **308,21 ms** simulados.
En el reloj pasaron **316,10 ms**.

La diferencia es **exactamente** lo que quedó en el acumulador: **7,89 ms**.

$$\text{simulado} + \text{acc} = \text{reloj}$$

## La divergencia del acumulador

250 ms de cuadro son **30 pasos** de golpe. Si eso alarga el cuadro siguiente, el siguiente pide más todavía. El tope **deja de simular en tiempo real a propósito** antes que congelarse.

## Si el acumulador se pusiera en cero

Se pierden **16,22 ms en cinco cuadros**, un **5 %**. Y no se recupera nunca: el reloj de la simulación se atrasa para siempre.

# Estado o evento: el flanco

Para usar el mouse como control de la cámara se propuso la estrategia de *polling de estados*.

Se agrega ahora una nueva estrategia para capturar eventos de entrada del usuario: *callback*.

| acción | naturaleza | mecanismo | ejemplos |
|---|---|---|---|
| **Control continuo** | estado | *polling* | alerones, timón, potencia, mover cámara |
| **Comando discreto** | evento | *callback* | pausar, alternar wireframe, cambiar de vista |

*Polling* contesta **"¿está apretada ahora?"**. Un *toggle* necesita **"¿se acaba de apretar?"**, y escrito con *polling* alterna 60 veces por segundo.

## Qué es un flanco

La transición entre dos estados es un **flanco**. Una forma de leerlo es esperar un $\Delta t$ antes de aceptar otro cambio. Sin embargo falla en dos casos: dos pulsaciones rápidas se pierden y una pulsación sostenida sigue cambiando, sólo que más lento. El *callback* de GLFW ya entrega el flanco: distingue `GLFW_PRESS` de `GLFW_REPEAT`.

# Cómo asignar las entradas de usuario a comandos del FDM

Los tres controles primarios del avión son ángulos **con signo** y el signo decide para qué lado se mueve el avión. Los ángulos son los que hay que vincular con las entradas del usuario.

| **mando** | **movimiento del comando** | **respuesta de la aeronave** | **deflexión** |
|---|---|---|---|
| Elevador | se empuja el bastón | baja la nariz | $\delta_e > 0$ |
| Alerones | bastón a la derecha | ala derecha abajo | $\delta_a > 0$ |
| Timón | pedal derecho | la nariz va a la derecha | $\delta_r < 0$ |

## Algo más que hay que decidir

¿Qué significa la flecha arriba del teclado? **Si es el bastón** —entonces la nariz baja, que es lo que hace un avión— o **si es la nariz** —y entonces sube, que es lo que espera quien no voló—. Ninguna de las dos es más correcta.

Hay que elegir una convención que sea coherente con lo que espera el usuario del simulador y la realidad que se está simulando.

# La proporcionalidad de los comandos y como abordar el problema

Hay un problema al usar el teclado como sistema de entrada para el control de la aeronave: una tecla es discreta —1 o 0— en cambio los comandos de la aeronave son proporcionales ($\delta = k \cdot s$).

**Solución óptima**: usar un joystick **proporcional** (palanca).

**Solución alternativa**: emular el comportamiento proporcional por software cuando las entradas son discretas.

| mapeo | forma matemática | estado | comportamiento |
|---|---|---|---|
| **Directo** | $\delta(t) = f(\text{teclas}(t))$ | sin memoria | discreto |
| **Rampa** | $\delta(t) = \delta(t_0) + \int r \cdot u(\text{teclas})\, dt$ | con memoria | proporcional |

```
// La rampa, adentro del update() del manejador de entrada:
if (arriba) elevator += rate * frame_dt;    // rate en [rad/s]
if (abajo)  elevator -= rate * frame_dt;
elevator = std::clamp(elevator, min_elevator, max_elevator);
```

Es el mismo esquema numérico de Euler del FDM pero usando el `dt` del cuadro.

# La condición inicial del integrador

La rampa para *emular* el comportamiento del movimiento del comando mediante teclado requiere de una condición inicial para empezar a integrar.

## Problema: el avión que se descompensa en el primer cuadro

El FDM arranca desde una condición **trim** —equilibrio— y este equilibrio incluye una deflexión de elevador de un **valor específico**. `InputHandler` arranca en **cero**: la tecla no se presionó todavía.

**Hay dos lugares guardando la posición de los mandos y sólo uno se inicializó**. En el primer cuadro se envía al FDM un comando de elevador que no es el del equilibrio y el avión se descompensa solo sin ninguna razón externa.

## Solución

Un método `set_controls()` en `InputHandler` que se llama **antes del loop** junto con el estado de trim. **Estado y comandos de trim son un solo dato repartido en dos lugares**: cambiar uno sin el otro rompe el equilibrio sin causa aparente.

# InputHandler: ampliación del módulo

```
class InputHandler {
public:
    void update(GLFWwindow* w, float dt);          // polling, 1 vez por cuadro
    const ControlInputs& controls() const;         // lo que consume el FDM
    void set_controls(const ControlInputs& c);     // la condicion inicial
    CameraCommand camera_cmd() const;              // el mouse, del practico anterior
};
```

## Qué hace

Traduce teclas y mouse a **números normalizados**, integra las rampas con el `dt` del cuadro y guarda la posición actual de los cuatro comandos de la aeronave.

## Qué **no** hace

No integra física, no dibuja, no conoce a `CameraSystem` y **no sabe qué es un alerón**: entrega cuatro números y un comando de cámara.

# Generación de la malla

El terreno se modela con una malla poligonal regular que se genera de manera automática (sencillo!).

Algoritmo para la grilla regular de vértices: cada vértice sale de los dos índices de los lazos que construyen la grilla.

```
// N celdas por lado  =>  N+1 vertices por lado.
for (unsigned i = 0; i <= N; ++i)
    for (unsigned j = 0; j <= N; ++j)
        vertices.push_back({ j * paso,  altura(i, j),  i * paso });
```

| | **fórmula** | $N = 200$ |
|---|---|---|
| vértices | $(N+1)^2$ | **40.401** |
| triángulos | $2N^2$ | **80.000** |

Los triángulos **duplican** a los vértices → cada vértice participa en varios triángulos a la vez.

¿Conviene indexar la malla?

# Por qué conviene que la malla esté indexada

Sin índices hay que listar un vértice por cada referencia: 80.000 triángulos necesitan **240.000** vértices. Cada vértice interior se repite **seis** veces.

## La regla para compartir vértices

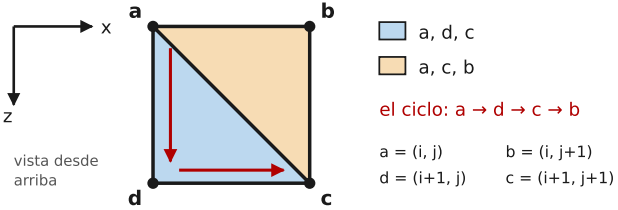
Dos vértices se comparten **si y sólo si coinciden en todos sus atributos**. En el cubo no se podía —cada esquina pertenece a tres caras de color distinto y de ahí los 24 vértices y no 8—. **En la grilla sí**: el vértice interior tiene una posición, una normal y un color y los seis triángulos que lo tocan usan esa misma tupla.

| $N = 200$ | **sin índices** | **con índices** |
|---|---|---|
| vértices en el buffer | 240.000 | **40.401** |
| memoria de vértices | 5,49 MiB | 969.624 B |
| memoria de índices | — | 960.000 B |
| **total** | **5,49 MiB** | **1,84 MiB** |

**5,94× menos vértices, pero 2,99× menos memoria**: el buffer de índices pesa casi lo mismo que el de vértices.

# Cómo ordenar los triángulos

Una celda de la grilla tiene **cuatro** esquinas y se dibuja con **dos** triángulos que comparten la diagonal: seis referencias a cuatro vértices.




```
indices.insert(indices.end(), {a, d, c});   // el ciclo correcto es a -> d -> c -> b
indices.insert(indices.end(), {a, c, b});
```

El orden de los índices es arbitrario

Escribirlo de otra manera **no produce ningún error**: compila, corre y dibuja un terreno. Pero puede tener consecuencias que vemos al final.

# Generación de los índices

El EBO guarda, para cada esquina de cada celda, **el número de orden de ese vértice** dentro del buffer. Con dos índices de bucle y una fila de ancho conocido ese número se calcula.

```
const unsigned fila = N + 1;                          // ancho en VERTICES, no en celdas
auto idx = [fila](unsigned i, unsigned j) { return i * fila + j; };

for (unsigned i = 0; i < N; ++i)                      // < N: se recorren CELDAS
    for (unsigned j = 0; j < N; ++j) {
        const unsigned a = idx(i, j),     b = idx(i,   j+1);
        const unsigned d = idx(i+1, j),   c = idx(i+1, j+1);
        indices.insert(indices.end(), {a, d, c});
        indices.insert(indices.end(), {a, c, b});
    }
```

Los dos bucles no tienen el mismo límite

El de **vértices** va `<= N` —hay $N+1$— y el de **celdas** va `< N`. Confundirlos no da un error de compilación: da un índice que apunta a otro vértice, o fuera del buffer y el malla se deforma sin causa aparente.

# Cómo darle forma al terreno

La malla del bloque anterior es un plano: la posición horizontal de cada vértice la fija el bucle y la altura es la misma para todos. Darle forma al terreno es **variar una sola coordenada por vértice**.

| coordenada | de dónde sale | varía |
|---|---|---|
| $x$, $z$ | de los índices del bucle, $j \cdot paso$ y $i \cdot paso$ | no: la grilla es regular |
| $y$ | del **dato de altura** | sí: es el terreno |

## Datos reales

Altimetría medida, publicada en grillas regulares —misiones satelitales, servicios de elevación—. Se baja una zona y se usa tal cual. Da un terreno **reconocible**.

## Generación automática

Una función que devuelve una altura para cada punto: ruido, sumas de senos, ruido de Perlin. Da un terreno **ilimitado y sin repetición visible**.

Las dos familias entregan **lo mismo**: un número por nodo de la grilla.
La malla no cambia; cambia la forma de obtener la altura.

# Los datos de altura: *heightmap*

Un *heightmap* es, literalmente, **una grilla de números**: una altura por nodo. No tiene geometría, ni triángulos, ni normales.

| **soporte** | **cómo se lee** | **rango** |
|---|---|---|
| Imagen en escala de grises | un canal por píxel | 0 a 255, o 0 a 65535 |
| Archivo de texto | un número por nodo | representación que tenga el archivo |
| Función evaluada | se llama con $(i, j)$ | limitada por el tipo de retorno |

Las alturas pueden **fijar al cargar** —se leen una vez y quedan escritas en el buffer de vértices— o **quedar en la GPU como textura** y leerse en el *vertex shader* cuadro a cuadro, que permite obtener un terreno que cambia (*displacement mapping*

# Ajustar la escala del terreno

La grilla tiene dimensiones genéricas que no representan el tamaño que queremos para la escena.

Los datos de altura (heightmap) pueden o no tener unidades.

Hay que ajustar el tamaño de la malla para que sea representativa de la escena tomando tres decisiones:

| **decisión** | **qué fija** | **si está mal** |
|---|---|---|
| Separación entre vértices | el detalle del relieve | terreno anguloso o malla muy fina |
| Escala vertical $s$ | la amplitud del relieve | terreno *picudo* o *planchado* |
| Tamaño de la grilla $N$ | la extensión del mundo | se alcanzan los límites (bordes) |

| **celdas** | **separación** | **lado** | **vértices** | **memoria** |
|---|---|---|---|---|
| 200 | 10 m | 2,0 km | 40.401 | 1,8 MiB |
| 400 | 25 m | 10,0 km | 160.801 | 7,3 MiB |
| 800 | 25 m | 20,0 km | 641.601 | 29,3 MiB |
| 800 | 50 m | 40,0 km | 641.601 | 29,3 MiB |

# Cómo se integra el terreno al simulador

La aeronave del proyecto vuela en condición de equilibrio a **150 m/s**. Como queremos representar en tiempo real tenemos que compatibilizar el tamaño del terreno con los datos de la aeronave.

| lado del terreno | memoria | tiempo en cruzarlo |
|---:|---:|---:|
| 2,0 km | 1,8 MiB | 13 s |
| 20,0 km | 29,3 MiB | 2 min 13 s |
| 40,0 km | 29,3 MiB | 4 min 27 s |

Agrandar la malla **posterga** el problema, no lo resuelve: más celdas es memoria al cuadrado, y más separación es menos detalle. En el mejor caso, cuatro minutos de vuelo.

## Repetir la misma malla

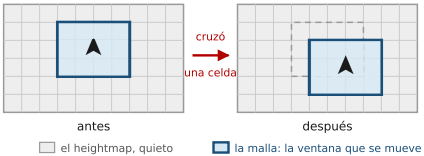
La misma malla dibujada varias veces con **distintas matrices de modelo**: $3 \times 3$ sobre 20 km dan 60 km de lado, 9 *draw calls* y la misma memoria.

## Fijar un límite y justificarlo

El borde pasa a ser **una regla del simulador**: aviso y vuelta, o fin del vuelo. Por ejemplo, en el simulador de acrobacia la aeronave debería volar en una zona reducida.

# Terreno infinito: heightmap como mundo y malla como región

Una estrategia es separar el dato de la geometría: el heightmap representa el mundo y es estático, mientras que la malla representa una región y se desplaza.




| | |
|---|---|
| **se construye una vez** | los índices y la topología de la ventana |
| **se recalcula al moverse** | sólo la **altura** de los vértices |
| **cuándo** | al cruzar una celda: con paso de 25 m, cada **0,17 s** |

La ventana se mueve de a una celda, no sigue al avión de forma continua

La aeronave se mueve libre **dentro** de la ventana; al cruzar una celda, la ventana se mueve un paso y vuelve a quedar centrada. Así cada vértice cae siempre sobre un nodo del heightmap y el relieve queda fijo.

# Activar eliminación de superficies ocultas

Dos mecanismos: el **z-buffer** que resuelve qué fragmento queda adelante y el **recorte de cara trasera** que descarta los polígonos que miran en otro sentido antes de rasterizarlos.

En OpenGL es muy simple usar estas funcionalidades: son **dos líneas** y las dos vienen apagadas por defecto.

```
glEnable(GL_DEPTH_TEST);    // z-buffer
glEnable(GL_CULL_FACE);     // recorte de cara trasera
```

## Problema: al activar la segunda desaparecen caras

Ejemplo: el terreno estaba dibujándose bien. Se agrega la línea del recorte de cara trasera y partes del terreno dejan de verse: a veces el terreno entero, a veces a parches.

No hay error de compilación y tampoco hay mensaje de OpenGL. El programa funciona y la imagen está mal.

¿Por qué una línea que sólo descarta triángulos puede hacer desaparecer el terreno?

# Cómo funciona en la práctica: OpenGL

Teoría: la normal exterior en sentido dextrógiro, si $LOS \cdot ON > 0$ el polígono es trasero y se descarta.

En OpenGL el atributo normal no participa de esta decisión

OpenGL toma el **signo del área del triángulo ya proyectado en pantalla**: si sus tres vértices quedan en sentido **antihorario**, es cara frontal. No mira el atributo `normal` del vértice que puede incluso no existir.

O sea: **decide por el orden en que se listaron los vértices.**

```
glFrontFace(GL_CCW);        // la convencion: antihorario es cara FRONTAL (es el default)
glCullFace(GL_BACK);        // que se descarta (es el default)
```

Por eso al ordenar los vértices de una malla poligonal (winding) para procesar **en OpenGL** hay que usar la convención *antihorario mirando la cara desde afuera*.

# Cómo diagnosticar problemas de visibilidad

Cuatro formas. Cada una descarta una familia entera, y las cuatro se contestan cambiando una línea.

| qué se hace | qué se mira | qué significa |
|---|---|---|
| `glPolygonMode(..., GL_LINE)` | ¿aparece el alambre? | Si no aparece, **el problema no es el recorte**: es la geometría, la cámara o la matriz |
| `glDisable(GL_CULL_FACE)` | ¿vuelven las caras? | Si vuelven es **winding**, no geometría |
| `glFrontFace(GL_CW)` | ¿se invierte el síntoma? | Si se invierte, es una **inversión global**: todo el bucle al revés |
| — | ¿los agujeros son **regulares**? | Entonces es **inconsistencia**, y el patrón dice qué parte del bucle la produce |

# Práctico 07 — El terreno en la escena

1. **La malla de terreno indexada**: la grilla generada con el doble bucle y las alturas leídas de un heightmap.
2. **Las normales de la malla**, con la fórmula en el enunciado.
3. **El recorte de cara trasera encendido**, y el terreno visible con el recorte activado.

## Dos cuestiones que hay que decidir

**Qué tamaño tiene el mundo** y **qué pasa al llegar al borde**: son decisiones del simulador y hay que escribirlas.

## Pregunta para la próxima clase

El terreno de hoy es de un solo color plano. Un terreno real tiene pasto, roca, nieve y sombra.
**¿De dónde sale el color de cada punto de una superficie?**